main

October 30, 2026

#

Deliverable 02

0.0.1 Task 1 - File Ingestion and reguar expressions

[44]:
```
from pathlib import Path

import re

import pandas as pd


import json

# Task 1.1
def read_log_file(file_path):
    path = Path(file_path)

    try:
        content = path.read_text(encoding="utf-8")
        return content, "utf-8"

    except UnicodeDecodeError:
        content = path.read_text(encoding="iso-8859-1")
        return content, "iso-8859-1"



# Task 1.2

def log_parser(log):
    log_lines = log.split('\n')
    logs = []
    pattern = r"\[(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\] \[(POS_\d+)\]␣
 ↪item=(\w+) price=(\d+\.\d+) (\w+) \((\w+)\)"

    for log_line in log_lines:
```

```
        match = re.match(pattern, log_line)
        if match:
            log_dict = {
                "timestamp": match.group(1),
                "register_id": match.group(2),
                "item": match.group(3),
                "price": float(match.group(4)),
                "currency": match.group(5),
                "status": match.group(6)
            }
            logs.append(log_dict)

    return logs
```

0.0.2 Task 2 - Recursive utility functions

[45]:
```
# Task 2.1

def find_log_files(directory):
    directory = Path(directory)
    log_files = []

    for item in directory.iterdir():

        if item.is_dir():
            log_files.extend(find_log_files(item))

        elif item.is_file() and item.suffix == ".log":
            log_files.append(item)

    return log_files


# Task 2.2

def find_active_registers(data) -> set[str]:
    active_registers = set()

    if isinstance(data, dict):

        if data.get("active") is True and isinstance(data.get("id"), str):
            active_registers.add(data["id"])

        for value in data.values():
            active_registers.update(find_active_registers(value))

    elif isinstance(data, list):
```

```
        for item in data:
            active_registers.update(find_active_registers(item))

    return active_registers
```

0.0.3 Task 3 - Functions for cleaning and analysing data

[46]:
```
import numpy as np

# Task 3.1

def remove_price_outliers(logs):
    prices = [d.get("price") for d in logs if "price" in d]

    mean_price = sum(float(price) for price in prices) / len(prices)
    std_dev = (sum((float(price) - mean_price) ** 2 for price in prices) /␣
 ↪len(prices)) ** 0.5
    median_price = np.median([float(price) for price in prices])

    lower_bound = mean_price - 3 * std_dev
    upper_bound = mean_price + 3 * std_dev

    cleaned_logs = []

    for log in logs:
        if float(log["price"]) > lower_bound and float(log["price"]) <␣
 ↪upper_bound:
            cleaned_logs.append(log)
        else:
            cleaned_logs.append(median_price)

    return np.array(cleaned_logs)

# Task 3.2

def compute_refund_ratio(sales_count, refund_count):
    epsilon = 1e-6
    refund_ratio = refund_count / (sales_count + epsilon)

    return refund_ratio
```

0.0.4 Task 4 - Data analysis using Pandas

[47]:
```python
# Task 4.1 creating dataframe
def create_dataframe(transactions):
    df = pd.DataFrame(transactions)

    df["timestamp"] = pd.to_datetime(df["timestamp"])

    df = df.set_index("timestamp")

    df = df.sort_index()

    return df

# task 4.2 revenue per item per hour
def hourly_revenue(df):
    revenue = (
        df.groupby("item")
        .resample("1h")["price"]
        .sum()
        .reset_index()
    )

    return revenue

# Task 4.3 Pivot table

def status_pivot_table(df):
    pivot = pd.pivot_table(
        df,
        values="price",
        index="item",
        columns="status",
        aggfunc="sum",
        fill_value=0
    )

    pivot = pivot.reindex(
        columns=["SUCCESS", "REFUND", "CANCELLED"],
        fill_value=0
    )

    return pivot

# Task 4.4 - Refund ratio per item

def item_refund_ratios(df):
```

```
    counts = (
        df.groupby(["item", "status"])
        .size()
        .unstack(fill_value=0)
    )

    counts = counts.reindex(
        columns=["SUCCESS", "REFUND"],
        fill_value=0
    )

    sales_count = counts["SUCCESS"].to_numpy()
    refund_count = counts["REFUND"].to_numpy()

    ratios = compute_refund_ratio(sales_count, refund_count)

    result = pd.DataFrame({
        "sales_count": counts["SUCCESS"],
        "refund_count": counts["REFUND"],
        "refund_ratio": ratios
    })

    return result
```

0.0.5 Task 5 - Create full workflow

[48]:
```
# Task 5 - Full workflow

def full_workflow(data_directory="data"):

    data_path = Path(data_directory)
    log_files = find_log_files(data_path)

    with (data_path / "store_config.json").open(
        "r", encoding="utf-8"
    ) as file:
        config = json.load(file)

    active_registers = find_active_registers(config)

    print("Log files found:", len(log_files))
    print("Active registers:", active_registers)


    transactions = []

    for log_file in log_files:
```

```
    content, encoding = read_log_file(log_file)
    parsed_logs = log_parser(content)

    for transaction in parsed_logs:

        if transaction["register_id"] in active_registers:
            transactions.append(transaction)

print("Transactions found:", len(transactions))

if not transactions:
    raise ValueError("No valid transactions found")



df = create_dataframe(transactions)



cleaned_logs = remove_price_outliers(transactions)

cleaned_prices = np.array([
    float(value["price"]) if isinstance(value, dict)
    else float(value)
    for value in cleaned_logs
], dtype=float)

df["price"] = cleaned_prices


nok_mask = df["currency"] == "NOK"

df.loc[nok_mask, "price"] = (
    df.loc[nok_mask, "price"] * 10.0
)



hourly = hourly_revenue(df)

pivot = status_pivot_table(df)

refunds = item_refund_ratios(df)
```

```
    df.to_csv(
        "cleaned_transactions.csv",
        index_label="timestamp"
    )

    hourly.to_json(
        "hourly_revenue.json",
        orient="records",
        date_format="iso",
        indent=2
    )

    pivot.to_csv(
        "status_pivot_table.csv"
    )

    refunds.to_csv(
        "item_refund_ratios.csv"
    )

    print("\nAll output files saved successfully!")

    return df


df = full_workflow()

print("\nFirst five transactions:")
print(df.head())
```

```
Log files found: 9
Active registers: {'POS_02', 'POS_03', 'POS_01'}
Transactions found: 720

All output files saved successfully!

First five transactions:
                    register_id        item  price currency     status
timestamp
2026-10-15 07:00:10      POS_02  Cappuccino  75.20      NOK  CANCELLED
2026-10-15 07:00:15      POS_03      Muffin  36.10      NOK    SUCCESS
2026-10-15 07:00:26      POS_01    Sandwich   3.08      USD    SUCCESS
2026-10-15 07:00:47      POS_03      Muffin   7.70      EUR    SUCCESS
2026-10-15 07:01:06      POS_02       Latte  74.00      NOK  CANCELLED
```